

Esempio colloquio

- Rifare la presentazione
- Decoratori → API (esempio di Unipol)
- Esempi - lista di account e un contenente figlio
- LWC
- Apex
- Trigger
- Eventi
- Before e after e cosa significa
- Visual force
- Chiamate API
- Crud
- Parte admin (custom object)
- Look up e master detailed
- Roll up summary
- Flow
- Obiettivo preparare una presentazione

quando spiego dire prima tutte le cose che so, esempio la parte dev
descrizione admin
page

Introduzione personale

Buongiorno a tutti, mi chiamo Matteo Vattimo, ho 23 anni sono un salesforce developer e ho una certificazione come platform developer 1. Da un anno sto lavorando su un progetto incentrato sullo sviluppo di LWC per un cliente in ambito assicurativo. La mia esperienza con la programmazione è però iniziata dalla scuole

superiore quando all'istituto tecnico che ho frequentato siamo stati introdotti a java e alla programmazione ad oggetti. Al termine delle superiori, incuriosito e interessato a questo mondo mi sono iscritto ad un ITS, un percorso formativo di specializzazione sulla programmazione. e sono un Junior Developer in Salesforce, sono in Codestorm da 6 mesi. Ho fatto un percorso formativo studiando con trailhead, conoscendo il mondo Salesforce. Ho fornito supporto ad alcune task dei miei colleghi. Ho intrapreso questo percorso professionale perché sono appassionato di informatica e mi piace risolvere problemi aiutando le aziende a migliorare i propri processi. Inoltre, ho dimostrato adattabilità durante il mio periodo formativo e ho conseguito la certificazione di Platform Developer I

Mi chiamo Matteo Vattimo e sono un ragazzo appassionato di programmazione dalle scuole superiori, tanto che al termine di queste mi sono iscritto ad un ITS focalizzato sulla programmazione ad oggetti tramite Java e front-end tramite HTML, CSS e JavaScript. Sono ormai 11 mesi che sono in Codestorm, ho fatto un percorso formativo studiando con Trailhead, conoscendo il mondo Salesforce e sono riuscito a prendere la certificazione di Platform Developer I in un paio di mesi. Attualmente sto lavorando per un cliente in ambito assicurativo con un focus esclusivamente su LWC per la realizzazione di pagine per il loro Contact Center.

Note per l'esposizione orale – Introduzione:

- **Durata:** ~1-2 minuti. Aprire presentandosi con nome, ruolo attuale ed esperienza in sintesi.
- **Focus:** Evidenziare la motivazione per cui si è scelto Salesforce e i propri punti di forza – includere competenze tecniche chiave e soft skill (es. comunicazione, problem-solving, adattabilità).
- **Suggerimenti:** Mantenere un tono sicuro e positivo. Se l'interlocutore è HR/non tecnico, evitare gergo eccessivo e sottolineare passione e capacità di collaborazione. Trasmettere entusiasmo genuino per il proprio lavoro.

Domanda frequente:

"Mi parli di Lei."

– Preparare una risposta che riassume il proprio percorso, le certificazioni ottenute e ciò che motiva il candidato in questo ruolo.

Consiglio prova colloquio

Competenze come Salesforce Developer (*sezione opzionale*)

Oltre all'esperienza da Admin, ho coltivato competenze da Salesforce Developer che mi permettono di estendere le funzionalità della piattaforma tramite codice e componenti personalizzati. La mia esperienza nello sviluppo comprende **[ESPERIENZA_DEV]**. In particolare, le mie skill tecniche di sviluppo includono:

Apex (classi, trigger e test)

Tramite Apex è possibile implementare logiche di business complesse. Le classi apex vengono usate principalmente per i Trigger, che permettono di collegare logica complessa al verificarsi di un evento su un record. Questo viene specificato nell'intestazione del trigger tramite keyword *before* o *after* e *insert*, *update* o *delete*. Si possono creare classi di test, Salesforce richiede che il codice apex abbia una copertura almeno del 75% delle righe garantendo copertura dei casi limite, inoltre tramite l'Assert è possibile specificare valore di input e valore aspettato, garantendo un controllo aggiuntivo anche nei casi limite.

Tendo sempre a strutturare il codice in classi modulari, separate dal trigger, seguendo il Trigger Handler Pattern: questo mi permette di rendere il codice manutenibile e facilmente testabile.

Quando sviluppo trigger, faccio sempre attenzione al principio di bulkification, lavorando su liste e mappe per gestire anche grandi volumi di record senza superare i limiti di Salesforce.

Ogni volta che scrivo logica applicativa, accompagno il codice con test class dedicate, creando dati sintetici di test e verificando sia i casi standard sia gli edge case.

Ad esempio, in un progetto formativo ho sviluppato un trigger su Opportunity che calcolava il margine e, quando superava una soglia, inviava automaticamente una notifica su Chatter ai manager. Ho gestito anche il caso in cui la notifica non doveva essere duplicata su record già segnalati.

Scrivo codice **Apex** per implementare logiche di business complesse lato server, andando oltre le funzionalità offerte dagli strumenti dichiarativi.

Ho esperienza nella creazione di **classi Apex modulari**, con metodi riutilizzabili, documentati e orientati alla manutenibilità. Strutturo la logica in modo che sia separata dalla logica di chiamata (es. Trigger Handler Pattern), per facilitare test e riuso.

Progetto **Apex Trigger** che rispondono a eventi come insert, update, delete, rispettando sempre i principi di **bulkification** (uso di loop su liste, query ottimizzate, uso di Map per aggregare dati) per garantire performance elevate anche in operazioni massive.

Per ogni componente di logica sviluppata, implemento **test class dedicata** con dati sintetici e System.assertEquals mirati, garantendo una **copertura >75%** e verificando la correttezza della logica anche in scenari edge-case.

Esempio pratico: in un progetto formativo, ho sviluppato un trigger su Opportunity che calcolava il margine previsto e inviava notifiche Chatter ai manager quando il valore superava una soglia definita, includendo una logica di filtro per record già notificati.

Lightning Web Components (LWC) e interfaccia utente

Sviluppo componenti **Lightning Web Components (LWC)** per arricchire l'esperienza utente in Salesforce Lightning Experience, seguendo un approccio modulare e riutilizzabile.

Utilizzo lo stack standard composto da **JavaScript moderno (ES6+)**, **HTML5 e CSS3**, in conformità allo **Salesforce Lightning Design System (SLDS)**.

Sono in grado di progettare componenti frontend che ricevono dati da Apex (via @wire o imperative call), espongono proprietà pubbliche (@api), gestiscono eventi custom e comunicano con altri componenti tramite event bubbling.

Integro i **Lightning Base Components** per costruire interfacce coerenti con lo standard Salesforce: ad esempio, uso lightning-datatable, lightning-input, lightning-card, lightning-button, lightning-toast per realizzare UI funzionali e accessibili.

In un esempio concreto, ho creato un componente AccountContactList che visualizza una tabella con i contatti associati a un Account, include filtro dinamico e sorting client-side. Il componente è stato pensato per essere riutilizzato su più record type, con configurabilità tramite attributi esterni.

Ho inoltre familiarità con **Visualforce** e **Aura Components**, per manutenzione o migrazione di interfacce legacy.

Integrazioni e API esterne

Sono in grado di realizzare **integrazioni point-to-point** tra Salesforce e sistemi terzi tramite **Apex callout** su servizi RESTful.

Utilizzo classi native come Http, HttpRequest e HttpResponse per inviare richieste **GET, POST, PATCH**, gestendo le intestazioni, i payload e l'analisi delle risposte in formato **JSON** o **XML**.

Includo sempre la gestione degli errori (try-catch, fallback logico) e tracciamento delle risposte con eventuale logica di audit o fallback.

Configuro correttamente i **Remote Site Settings** per autorizzare domini esterni, e quando necessario registro Named Credential per semplificare l'autenticazione.

Esempio concreto: ho sviluppato una funzione di enrichment indirizzo per il Lead, che al salvataggio esegue un callout a Google Maps API e scrive latitudine e longitudine in campi custom. La chiamata è asincrona (futura implementazione con Queueable in roadmap) e dotata di gestione completa dei casi d'errore e logging in oggetto Integration_Log__c.

Questa capacità di **integrare dati in ingresso e in uscita** amplia l'interoperabilità di Salesforce, rendendolo un nodo centrale dell'architettura applicativa.

Strumenti di sviluppo e deployment

Utilizzo **Visual Studio Code** con plugin **Salesforce Extension Pack** per un'esperienza di sviluppo completa, con supporto a IntelliSense, debugging e CLI integrata.

Sono pratico nell'uso di **Salesforce DX (SFDX)** per gestione dei metadata, sviluppo in scratch org, versioning e deploy controllato.

Adotto **Git** per versionamento del codice (branch feature, merge, pull request, gestione conflitti), facilitando la collaborazione in team e il tracciamento puntuale delle modifiche.

Per la fase di deployment, utilizzo sia **Change Set** per org non-DX, sia metodi alternativi (es. sfdx force:source:deploy) per pipeline automatizzate. So testare e validare l'integrità del pacchetto prima del rilascio.

Sono consapevole delle **best practice DevOps**, anche se non ho ancora esperienza diretta su CI/CD in ambiente Salesforce (es. GitHub Actions, Copado o Bitbucket Pipelines).

Questa organizzazione dello sviluppo garantisce **qualità, ripetibilità e tracciabilità** nel ciclo di vita del software Salesforce.

Note per l'esposizione orale – Competenze Developer:

- **Durata:** ~3-5 minuti. Strutturare anche questa sezione in sottoparti (codice Apex, sviluppo UI, integrazioni, etc.), dedicando circa 1 minuto a ciascun argomento rilevante per il proprio profilo di sviluppatore.
- **Focus:** Mettere in evidenza le competenze di programmazione su Salesforce dimostrando **padronanza tecnica e capacità di risolvere problemi complessi**. È importante fornire esempi: ad ogni competenza menzionata associare uno scenario concreto (es. "ho sviluppato un trigger che...", "ho creato un componente LWC per...") evidenziando come quell'intervento ha risolto un problema o migliorato il sistema. Far emergere anche la conoscenza delle best practice (es. citare la gestione dei *governor limits*, l'uso di design pattern come il trigger handler, l'attenzione alla copertura di test).
- **Suggerimenti:** Mostrare sicurezza nel linguaggio tecnico, usando i termini corretti (Trigger, LWC, API REST, ecc.) per indicare familiarità con l'ecosistema Salesforce. Allo stesso tempo, spiegare in modo comprensibile **l'impatto** del lavoro di sviluppo: ad esempio, come una certa integrazione ha permesso di evitare inserimenti manuali, o come un componente UI ha reso più efficiente un processo per gli utenti. Questo aiuta anche chi ascolta (tecnico o non tecnico) a capire il valore concreto delle proprie abilità da developer. Se si hanno conoscenze aggiuntive (es. DevOps, CI/CD, altri linguaggi) e sono rilevanti per il ruolo, è utile accennarle brevemente.

Domande frequenti:

"Mi descriva un progetto di sviluppo Salesforce a cui ha lavorato e le sfide tecniche affrontate."

,

"Quali best practice segue quando scrive codice Apex?"

,

"Ha esperienza di integrazione di Salesforce con sistemi esterni? Ci può fare un esempio."

– Preparare risposte che raccontino un progetto specifico (es. sviluppo di una funzionalità custom o integrazione) evidenziando il proprio ruolo, le soluzioni adottate e i risultati ottenuti. Aspettarsi anche domande tecniche approfondite su Apex e LWC (es. gestione degli errori, sicurezza del codice, performance) se il colloquio è con un team tecnico.

Competenze come Salesforce Administrator (*sezione opzionale*)

La mia esperienza come Salesforce Administrator comprende **[ESPERIENZA_ADMIN]**. In particolare, le mie competenze da Admin includono:

Gestione e personalizzazione degli oggetti

Salesforce è composto principalmente da oggetti, questi possono essere standard come

- Account: Azienda
- Contact: Persona collegata, come un dipendente
- Opportunity: Potenziale vendita
- Lead: potenziale cliente, non ancora qualificato
- Case: richiesta di support
- Product: Prodotto
- Quote: Preventivo

- Quote Line: righe con prodotti, quantità e prezzi. Relazione Master-Detail con Quote

Quando le richieste del cliente non possono venir soddisfatte da questi oggetti, si ricorre agli oggetti custom. In questi casi il nuovo oggetto potrà venir personalizzato con campi di tipo

- Testo, numero, data, currency, boolean
- Picklist aggiungendo una lista di valori tra cui scegliere
- Formula dove appunto è possibile aggiungere una formula per calcolare a runTime un valore
- Relazioni, lookup relationship oppure master-detail relationship

Posso quindi impostare dei page layout, che mi permettono di personalizzare la visualizzazione dell'oggetto

Inoltre posso specificare regole di accessibilità all'oggetto tramite profili e permission sets oppure tramite ruoli e sharing rules ai record

Ho consolidato esperienza nella configurazione e manutenzione degli **oggetti standard** di Salesforce, come Account, Contatti, Lead, Opportunità e Case, adattandoli ai processi aziendali specifici tramite configurazione di campi personalizzati, layout e logiche di business.

Quando le esigenze aziendali non possono essere soddisfatte con gli oggetti predefiniti, progetto e implemento **oggetti custom** coerenti con il data model esistente. In questi casi definisco le relazioni appropriate (lookup, master-detail), imposto regole di condivisione, creo validazioni per garantire l'integrità dei dati e documento il design funzionale.

Ad esempio, in un progetto accademico ho realizzato un oggetto Project__c relazionato agli Account, dotato di campi custom per tracciare stato, responsabile interno, scadenze e metriche. In fase di progettazione ho seguito le best practice per denominazione dei campi, uso dei record type e applicazione di standard UX coerenti.

Interfaccia utente e pagine Lightning

Sono in grado di personalizzare l'esperienza utente in Salesforce Lightning sfruttando sia i **Page Layout** tradizionali, sia il **Lightning App Builder** per creare Lightning Record Page dinamiche e ottimizzate per ruolo/profilo.

Gestisco i Page Layout per definire visibilità e disposizione dei campi, sezioni, related list, pulsanti e quick actions in funzione del profilo utente.

Utilizzando il Lightning App Builder, progetto **Record Page su misura** strutturate su più colonne, con componenti standard (es. report chart, related list filtered, rich text) e logiche di visibilità condizionale per mostrare informazioni pertinenti in base a criteri dinamici.

In un caso pratico, ho realizzato una Lightning Page Account per il team Sales, evidenziando lo stato dell'account con componenti visivi, inserendo un banner di alert in caso di clienti critici e separando le informazioni finanziarie da quelle operative tramite tab e sezioni personalizzate.

Controllo di sicurezza e accessi

Gestisco in modo approfondito la **Security Model** di Salesforce, garantendo che ogni utente acceda solo alle informazioni e funzionalità pertinenti.

Imposto e mantengo **profili utente e ruoli gerarchici**, bilanciando i permessi in base alla struttura organizzativa. Utilizzo i **Permission Set** per assegnare privilegi aggiuntivi in modo granulare e scalabile.

Configuro regole di **condivisione (sharing rules)** per estendere la visibilità dei record laddove necessario, mantenendo coerenza con le policy aziendali.

In ambienti più complessi, mi occupo anche di **criteri di accesso condizionali** e test dei permessi per garantire la separazione tra team (es. vendite e assistenza clienti). In un esempio reale, ho implementato un modello ibrido che limitava l'accesso ai record di Opportunità tra regioni diverse ma consentiva visibilità aggregata a supervisor di area.

Automazione dei processi

Progetto e implemento automazioni **declarative** per ridurre errori manuali e incrementare l'efficienza operativa.

Utilizzo **Flow Builder** per creare flow record-triggered, screen flow e autolaunched flow in grado di gestire logiche condizionali, aggiornamenti di dati, creazione di record, invio notifiche o chiamate a componenti Apex (invocable methods).

Gestisco **Process Builder** per orchestrare sequenze di azioni in scenari legacy o intermedi.

Creo e mantengo **Approval Processes** multistep per gestire logiche autorizzative strutturate (es. sconti sopra soglia, validazione documenti).

In un esempio significativo, ho automatizzato il processo di gestione ferie tramite flow e approval process, integrando notifiche email, aggiornamento dello stato della richiesta e blocco della modifica in caso di rifiuto.

Reportistica e dashboard

Costruisco **report personalizzati** e **dashboard interattive** per fornire visibilità sui KPI e facilitare decisioni data-driven.

Conosco la differenza tra i vari tipi di report (tabellari, summary, matrix, joined) e configuro **report type customizzati** quando i modelli standard non sono sufficienti.

Applico filtri, formule e bucket field per raffinare l'analisi dei dati. Le dashboard che progetto sono dinamiche, filtrabili per utente o team, e ottimizzate per la fruizione su dispositivi mobili.

Ad esempio, ho costruito una dashboard per il reparto vendite che aggregava le opportunità per area geografica, fase di vendita e valore potenziale, rendendo immediata l'analisi dell'imbutto commerciale e l'identificazione delle criticità.

Note per l'esposizione orale – Competenze Admin:

- **Durata:** ~3-4 minuti. È consigliabile suddividere l'esposizione in sottotemi, dedicando ~30-60 secondi a ciascun aspetto chiave.
- **Focus:** Illustrare le principali responsabilità tecniche da Admin accompagnandole con **esempi pratici**. Ad ogni competenza elencata dovrebbe corrispondere un breve caso concreto o risultato ottenuto (es: un flusso implementato che ha automatizzato un processo, un oggetto custom creato per un'esigenza specifica, ecc.).

- **Suggerimenti:** Mantenere un tono sicuro ma accessibile. Evitare di fare un elenco freddo di strumenti: meglio spiegare **come** ciascuna competenza ha avuto impatto positivo sul lavoro o sui risultati aziendali. Ad esempio, invece di dire solo "so creare Flow", si può aggiungere *"ho automatizzato il processo X con un Flow, riducendo i tempi di Y%"*. Questo rende la presentazione più coinvolgente e memorabile. Adeguare il livello di dettaglio all'interlocutore: con un esaminatore tecnico si può entrare di più nei particolari, mentre con HR conviene restare su benefici e panorama generale.

Domande frequenti:

"Ci descriva un progetto o una soluzione che ha implementato come Admin."

,

"Come ha gestito [una sfida tipica, ad es. la pulizia dei dati o la gestione di permessi] nella sua esperienza?"

. – Preparare uno o due esempi concreti che mettano in luce problem-solving e padronanza degli strumenti dichiarativi (ad es. racconto di un'automazione ben riuscita o di come si è migliorata la qualità dei dati con regole e controlli).

fields, page layout, validation rule

Domande Possibili lato Admin

*"Se un utente **non vede un record**, quali sono **i principali elementi** di Salesforce che controlleresti per capire il problema?"*

La visibilità dei record dipende principalmente dagli **Sharing Settings (OWD)** dell'oggetto. Se l'oggetto è Private o Read Only, allora entrano in gioco **ruoli**, **gerarchia dei ruoli** e **sharing rules** per estendere la visibilità, inoltre controllerei anche se il record è assegnato all'utente, e se ci sono **permission set** che influiscono sull'accesso.

*"Se un campo è **presente nel Page Layout**, ma l'utente **non lo vede**, qual è la prima cosa che controlleresti?"*

La prima cosa che controllerei è il profilo e i permission set

"Qual è la differenza principale tra Profilo e Permission Set"

Il profilo definisce i permessi base di un utente su oggetti, campi e funzionalità, ed è uno solo per utente.

I permission set servono ad aggiungere permessi extra in modo modulare, senza dover creare nuovi profili, e un utente può averne più di uno.

"Quando useresti una Validation Rule invece di un Flow?"

Le Validation Rule servono per impedire il salvataggio di un certo dato quando non rispetta le condizioni. Invece i Flow serve per creare automazioni o delle azioni automatiche, esempio aggiornare campi o creare dei record

"Quando useresti una Queue invece di assegnare il record direttamente ad un utente?"

"Useresti una Queue quando un record deve essere gestito da un team e non da un utente specifico. La queue permette a più utenti di vedere il record e prenderlo in carico quando sono disponibile

"Cos'è una Flexipage?"

Una Flexipage è una pagina Lightning configurabile con l'App Builder che serve a comporre l'interfaccia utente, mostrando componenti standard o custom, e può essere usata come Record Page, App Page o Home Page

"Qual è la differenza tra Record Page, App Page e Home Page?"

La Home Page è una pagina di ingresso personalizzabile, spesso usata per mostrare report, dashboard e informazioni generali.

La **Record Page** è legata a un oggetto specifico e viene usata per visualizzare e lavorare sui singoli record di quell'oggetto.

La **App Page** è una pagina personalizzata pensata per uno scopo specifico all'interno di un'app, ad esempio per mostrare componenti custom o strumenti dedicati a un team.

*"Hai un oggetto **Case** (o **Ticket__c**) e vuoi che:*

- *quando viene creato un nuovo record*
- *venga **assegnato automaticamente** al team corretto*

Quale funzionalità useresti come admin e perché?"

Useresti un Record-Triggered Flow facendolo partire come Before save, facendo dei controlli sui campi utili come Category e Country, per poi impostare

l'OwnerId ad una Queue oppure uno User specifico

Un utente dice: "Io non vedo il campo `CodiceFiscale__c` sulla pagina di Contact, ma un mio collega lo vede".

Inizio controllando la Field-Level Security tramite profilo o permission set. Se il campo è visibile lì, controllo il page layout associato al record type dell'utente. I problemi di campo non dipendono dagli OWD, che riguardano solo la visibilità dei record.

Due utenti lavorano sullo stesso oggetto `Ordine__c`.

- Utente A **vede** il campo `Sconto__c`
- Utente B **non vede** il campo `Sconto__c`

Il record type non gestisce la visibilità del campo a livello di sicurezza. Prima controllerei la Field-Level Security (profilo/permission set). Se lì è ok, allora verifico il page layout e solo dopo se il record type porta a un layout diverso.

Puoi spiegarmi l'architettura di OmniStudio e il ruolo dei suoi componenti principali?

"OmniStudio è una suite low-code di Salesforce per creare esperienze guidate e integrazioni.

La parte frontend è composta da FlexCard, per la visualizzazione dei dati, e OmniScript, per la gestione dei flussi utente.

La parte backend è gestita tramite DataRaptor per il CRUD su Salesforce e Integration Procedure per orchestrare logica e integrazioni REST.

Tutti i componenti lavorano su un JSON che rappresenta lo stato dei dati

Qual è la differenza tra DataRaptor Extract e Turbo DataRaptor, e quando useresti uno o l'altro?

Il DataRaptor Extract permette di lavorare su più oggetti e supporta trasformazioni e formule.

Il Turbo Extract è più limitato: lavora su un singolo oggetto, non supporta formule né logica, ma è molto più performante.

Uso il Turbo quando devo solo leggere dati velocemente e non ho bisogno di trasformazioni.

Perché in OmniStudio è preferibile usare Integration Procedure per le integrazioni REST invece di chiamare direttamente Apex?

In OmniStudio si preferisce usare Integration Procedure perché è più performante, configurabile e manutenibile. Permette di orchestrare chiamate REST, DataRaptor e Apex senza scrivere codice, ed è ottimizzata per gestire grandi volumi di dati. Uso Apex solo quando serve logica molto custom o non supportata da OmniStudio

Come passi i dati da una FlexCard a un OmniScript

Le FlexCard gestiscono i dati tramite uno state JSON.

Quando si lancia un OmniScript, è possibile passare questo JSON — o una sua parte — come input tramite un'action.

L'OmniScript riceve il payload come stato iniziale e può usarlo per precompilare campi o gestire la logica del flusso

Cosa sono i limiti in Salesforce e a cosa servono?

I governor limits sono limiti imposti da Salesforce sull'esecuzione del codice Apex per garantire un utilizzo equo delle risorse in un ambiente multi-tenant, dove più organizzazioni condividono la stessa infrastruttura. Servono quindi a evitare che un singolo processo consumi troppe risorse e impatti negativamente gli altri utenti.

Questi limiti si applicano per ogni transazione Apex, quindi ad esempio nei trigger, nelle classi invocate da Flow, nelle chiamate API e anche nei test. Alcuni esempi comuni sono il limite di 100 query SOQL per transazione, 150 operazioni DML, circa 10 secondi di CPU time nelle esecuzioni sincrone e un massimo di 100 callout HTTP.

Se uno di questi limiti viene superato, Salesforce genera un errore runtime non gestibile, quindi è fondamentale scrivere codice ottimizzato. Per questo si applicano best practice come la bulkification, evitando query o DML dentro i loop, utilizzando collection come List, Set e Map, e quando necessario spostando la logica in asincrono con Queueable o Batch.

In sintesi, i governor limits influenzano direttamente il modo in cui si sviluppa in Apex, soprattutto in scenari come trigger su molti record o integrazioni, dove è importante progettare il codice per essere scalabile e rispettare questi vincoli

